

Accessible Multi-Factor Authentication for Visually Impaired Users: Unified Design

Group	Cryptix	Module	CS-3053 Computer Security (Semester 5)
Submission	2 (Combined Design)	Date	2026/09/21

Member	Student ID	Name with initials	Part of the system
M1	230135A	Dewdunika D.R.K.W.M.V.V	Accounts and Factor 1 (password / PIN, sessions)
M2	230279R	Jayasinghe E.A.T.D	Factor 2: authenticator-app codes, emailed codes, backup codes
M3	230384J	Madhubhashini M.D.D.	Factor 2: passkeys / security keys, remembered browser
M4	230562E	Samarakoon S.M.S.G.	Accessible client (screens, screen-reader support, speech)
M5	230684E	Wansandi M.G.J.	Orchestration, lockout, audit log, security evaluation

1. Introduction

1.1 Problem

Most login systems assume that the user can see. They show a QR code to scan, a picture puzzle to solve, a message in red, or a countdown timer. A blind or low-vision user who relies on a screen reader is either locked out or must ask a sighted person for help, which means handing over a secret. At the same time, a password on its own is a weak protection: it can be guessed, phished or reused from another website.

1.2 Purpose

The project task is: "*Design a multi-factor authentication system for visually impaired users.*" This report has to show two things:

- A. The system is multi-factor. A normal login requires two different kinds of proof: something the user *knows* (a password or PIN) and something the user *has* (a device with an authenticator app, or a passkey / security key). Knowing the password alone is not enough.
- B. The system can be used without sight. Registering, setting up the second factor, logging in and recovering an account can all be completed with a screen reader, a keyboard and audio. Nothing depends on seeing the screen.

The design also has to resist the attacks a login system faces: password guessing, replay, man-in-the-middle, cut-and-paste and interleaving attacks, session misuse, and skipping of the second factor. §6 explains each attack and how the design prevents or reduces it.

This document combines the five individual designs into one system. Where two members designed the same thing differently, the decision taken is recorded (§9 and Appendix A).

1.3 Target users

- Blind users who work with a screen reader (NVDA, JAWS, VoiceOver, TalkBack) and hear the page as speech.
- Braille-display users, who read the page as text on a braille line and may have speech switched off.
- Low-vision users, who use magnification, large text and high contrast.
- Users who control the computer by voice, who speak the names of buttons.

Fewer than 15 % of blind people read braille [12], so audio is the primary channel, but every message is also present as text so that braille users and deaf-blind users are not excluded.

1.4 How accessibility shaped the design

Accessibility is not a separate feature added at the end; it decided several parts of the security design:

- The second factor is offered as a passkey / security key or an authenticator app, and the app secret is never given as a QR code alone, because scanning a QR code normally requires sight (§7.6).
- Backup codes are digits only, because some letters sound alike when a screen reader speaks them.
- A login has exactly two steps, because each extra step is a real cost for someone who moves through a page one element at a time.
- There are no timers on any step, because a screen-reader user needs more time to find and fill a field, and the one-time codes already expire on their own.

- Secrets are never read aloud automatically, because a spoken secret can be overheard. A silent alternative always exists.

§7 gives the full accessibility design and the reason for each choice.

1.5 Structure of this document

§2 lists the requirements and §3 the architecture. §4 explains the authentication design and §5 shows how registration, login, lockout and recovery run. §6 is the security analysis. §7 is the accessibility design and §8 explains how passwords and other data are protected. §9 lists the main design decisions, §10 the planned tests, §11 the limitations and future work, and §12 concludes. Appendix A records how the five individual designs were combined and Appendix B summarises the API.

1.6 Terminology

Term	Meaning in this document
Authentication	The user proves their identity to the server, and the server proves its identity to the user (through its certificate) so that the user is not talking to an impostor.
Factor	A kind of proof: something the user <i>knows</i> (password, PIN) or something the user <i>has</i> (a device with an authenticator app, a passkey / security key, a saved backup code).
Multi-factor authentication (MFA)	Two different kinds of proof are required, so one stolen secret is not enough.
One-time password (OTP)	A code that is valid once. Here it is the 6-digit code shown by an authenticator app, which changes every 30 seconds, or an 8-digit code sent by email.
Passkey / security key	A private key kept on the user's phone, laptop or USB key. The device signs a challenge from the server; the server checks the signature with the matching public key. The private key never leaves the device.
Backup code	One of ten long codes given to the user at set-up, each usable once, for when the phone or key is lost.
Session	The record the server keeps after login so that it knows who is making later requests. The browser holds a random session ID.
Nonce / challenge	A fresh random value the server issues for one authentication run. The answer proves that it was made <i>now</i> , for <i>this</i> run, and not copied from an earlier run.
Hash	A one-way function. A stored hash can be compared with a new value, but the original value cannot be recovered from it. Used for passwords.
Encryption	A reversible transformation with a key. Used only where the server must later recover the original value.
Digital signature	A value that only the holder of a private key can produce and anyone with the public key can check. Used by passkeys and by the server's certificate.
Replay attack	Re-sending a captured valid message so that it is accepted a second time [9].
Man-in-the-middle attack	An attacker positioned between the two parties who reads, alters or inserts messages [9].
Cut-and-paste attack	Taking a valid part of one protocol run and inserting it into another so that the server accepts the combination [9].
Interleaving attack	Running two protocol sessions at the same time and using messages from one to answer the other [9].
Screen reader	Software that presents the screen as speech or braille.

1.7 Assumptions

#	Assumption	Why it is made	If it does not hold
A1	Users are blind or have low vision and use a screen reader or braille display every day.	The system is built to work <i>with</i> these tools, which most screen-reader users already have [10].	A telephone voice channel would be needed; out of scope.
A2	The client is a web page in a current browser. The user's phone or laptop supports passkeys or can run an authenticator app.	Screen readers already work in browsers; passkey support is built into current operating systems [3].	A user with neither device has no strong second factor and can only use the weaker email route (§5.4).
A3	The server has an HTTPS certificate from a trusted certificate authority. In the local demo the certificate is self-signed.	This is how a browser authenticates a server: the server's public key is in a certificate that the browser can verify [8].	Every secret could be read in transit. The whole security design depends on this assumption (§6.3).
A4	One server and one database, at coursework scale.	The project is a design and prototype, not a production service.	Running on several servers would need shared session and lockout data.
A5	Encryption keys are kept in a protected file on the server; there is no hardware key store.	No such hardware is available in a university project.	A full compromise of the server exposes the keys (§11).

#	Assumption	Why it is made	If it does not hold
A6	Every user has an email account they can read with their own tools. Email may take minutes to arrive.	Email is needed to confirm the address at registration and for recovery.	Email recovery stops working; the user still has the app, a passkey or backup codes.
A7	The user's own device, browser and screen-reader add-ons are not malicious.	A rogue browser extension can read anything the user can, and the server cannot prevent it.	Secrets typed on the device could be captured. The passkey's private key is the one secret it cannot read.
A8	Checking who a person is in the real world (identity proofing) is out of scope.	The task asks for a login system.	This is a scope limit, not a security claim.
A9	Everything the system speaks is also present as text on the page, and the user's voice is never used to identify them.	Braille users and deaf-blind users read text; voice identification would need its own security design.	Those users would be excluded, so this is a firm rule.

The design deliberately does not assume that a sighted helper is available, that the user can scan a QR code, that the user can hear, that any device can be trusted without evidence, or that the network is private.

2. System Requirements

2.1 Functional requirements

ID	Requirement	Owner
FR-1	A user can register with a username, an email address, and a password or a numeric PIN. The email address is confirmed with an emailed code before it is used for recovery.	M1, M5
FR-2	Every normal login needs Factor 1 (password or PIN) plus one strong second factor: a passkey / security key or an authenticator-app code.	M5
FR-3	A user can set up an authenticator app and one or more passkeys, can report any of them lost (it stops working at once), and can remove them.	M2, M3
FR-4	A user receives ten single-use backup codes and is told how many remain.	M2
FR-5	A user who has lost the second factor can log in with the password plus a backup code. A user with no second factor and no backup code can use an emailed code, but only into a restricted session that may set up a new factor after a 24-hour waiting period.	M2, M5
FR-6	A user who forgets the password can reset it with an emailed code. The second factor is still required afterwards.	M5
FR-7	After a full login the user may choose to remember the browser for a limited period. This is a convenience, not a factor.	M3, M5
FR-8	Every login, set-up, failure, lockout and recovery event is written to the audit log.	M5

2.2 Security requirements

ID	Requirement
SR-1	All communication uses HTTPS. The server refuses plain HTTP.
SR-2	The server must not allow a user to skip the second authentication step. Every request is checked against the state the server holds for that session.
SR-3	Every one-time value (app code, backup code, emailed code, passkey challenge) is accepted at most once.
SR-4	Every challenge is bound to the session and the purpose it was issued for, and expires after a short time.
SR-5	Passwords and backup codes are stored only as salted hashes. Secrets that must be recovered (the app secret, the email address) are stored encrypted, with the keys outside the database.
SR-6	Wrong attempts are limited: a growing lock after 5 wrong attempts in a row, and a hard lock after 100.
SR-7	The session ID is random, is replaced after each authentication step, expires after 30 minutes of inactivity or 12 hours in total, and is deleted on the server at logout.
SR-8	Error messages and response times do not reveal whether a username exists or which factors it has.
SR-9	Sensitive changes (adding or removing a factor, changing the password or email) need the second factor to be proven again within the last 5 minutes.
SR-10	The emailed-code route can never add a new second factor immediately; a 24-hour waiting period with a notification and a cancel link is required.

2.3 Accessibility requirements

ID	Requirement
AR-1	No step requires sight: no QR-code-only step, no visual CAPTCHA, no instruction given by colour alone.
AR-2	No step requires the user to memorise or retype a code by hand: paste and autofill always work, and the app secret is available as a link, as copyable text and as speech.
AR-3	Every screen works with the keyboard alone, in a sensible order, with no single-letter shortcuts.
AR-4	Every result is announced by the screen reader and also stays on the page as text, so braille users receive it.
AR-5	Every error is a full sentence that says what happened and what to do next.
AR-6	There are no countdown timers and no screens that move on by themselves. The only time limits are those built into the codes, and the session warns before it expires.
AR-7	Secrets are never read aloud unless the user asks, and a silent alternative always exists.
AR-8	Large text and high-contrast modes are available, and every button is large enough to use with an unsteady pointer.
AR-9	No action requires drag-and-drop or a mouse; every action is a labelled button.

The accessibility design follows the WCAG 2.2 guidelines at Level AA [4]. Conformance is not claimed until the planned tests in §10 have been completed on the implementation.

3. System Architecture

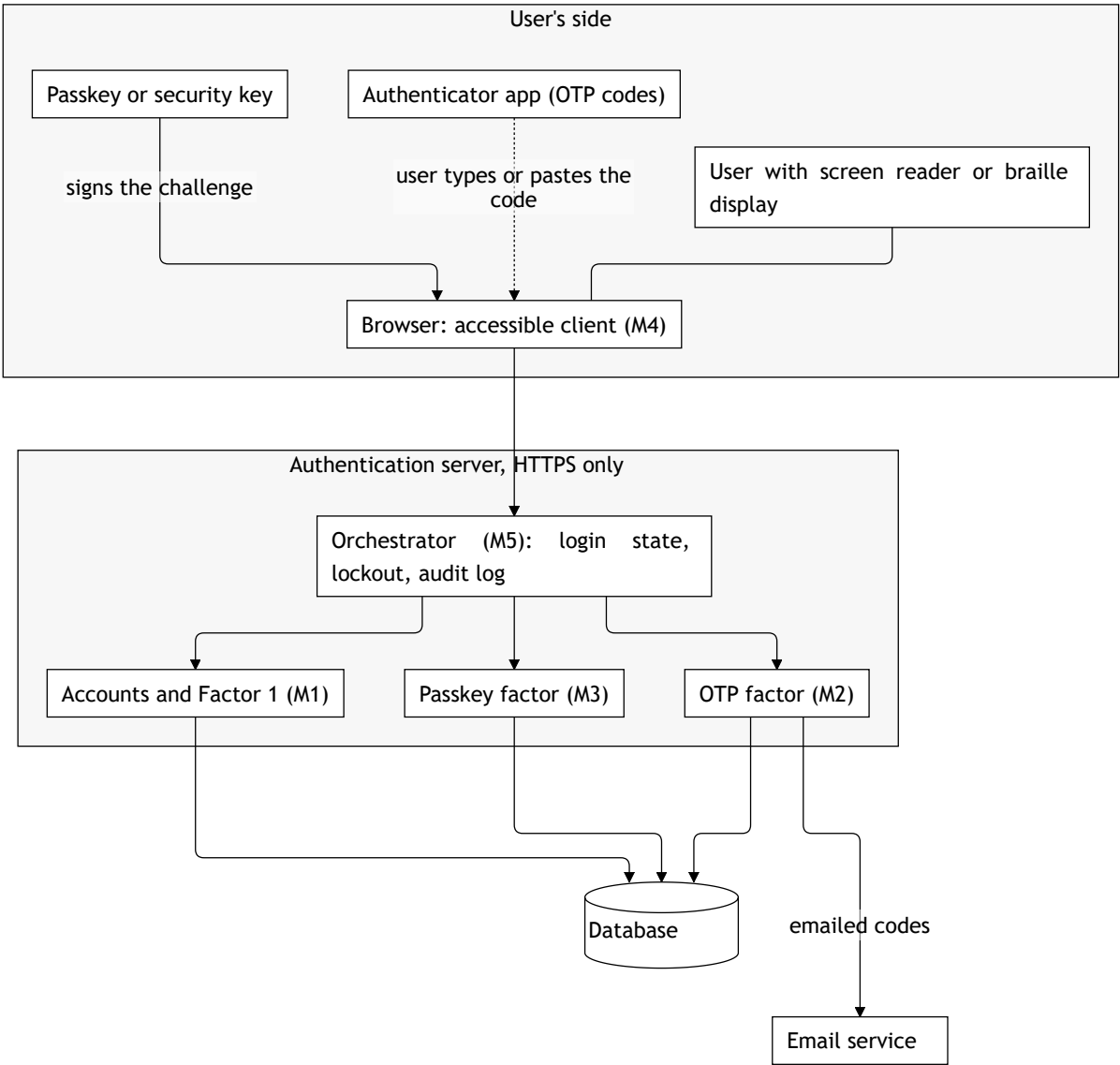


Figure 1. System architecture. The browser is the only client. All requests pass through the orchestrator, which holds the login state of every session and decides which step comes next; the three factor modules only check proofs and report the result. The authenticator app has no connection to any other component: it computes a code from its own clock, and the *user* transfers the code to the browser by typing or pasting. The passkey, by contrast, communicates with the browser directly and signs the server's challenge; the user only confirms.

3.1 Components and responsibilities

Component	Owner	Responsibility
Accessible client	M4	Screens, screen-reader announcements, keyboard operation, optional speech output, large-text and high-contrast modes
Accounts and Factor 1	M1	Registration, password/PIN hashing and checking, session records
OTP factor	M2	Authenticator-app set-up and code checking, emailed codes, backup codes
Passkey factor	M3	Passkey registration and login, remembered browser
Orchestrator	M5	Login state machine, lockout, audit log, recovery

The technology is Python with the FastAPI web framework, a PostgreSQL database, and a plain HTML and JavaScript client with no framework, because screen readers work best with ordinary HTML controls. All cryptography comes from established libraries; no cryptographic code is written by the team.

4. Authentication Design

4.1 The MFA concept

Factor 1 is something the user knows, such as a password or PIN. Factor 2 is something the user has, such as a device containing an authenticator app or a passkey / security key. The two factors provide different types of proof, so knowing the password alone is not enough to complete a normal login.

The system uses two different authentication factors, which gives stronger authentication than a password alone. NIST's digital identity guidelines describe this combination as their second assurance level (AAL2) [1]. This report does not claim formal compliance with that standard.

4.2 Factor 1: password or PIN

The user chooses a password of at least 8 characters or a numeric PIN of at least 6 digits. Both are checked against a list of common leaked passwords at registration, and both are stored only as a salted hash (§8.1). A PIN is offered because it can be typed on a telephone-style keypad, which many blind users know by touch; it is checked on the server exactly like a password, with the same lockout rules.

4.3 Factor 2: authenticator app or passkey

Option	Set-up	Proof at login	Why it proves possession
Passkey / security key (M3)	The device creates a key pair. The server stores the public key.	The server sends a fresh challenge. The device signs the challenge together with the website address, and the server verifies the signature with the stored public key.	Only the device holding the private key can produce the signature. This is public-key authentication with a digital signature [3]. Because the signature includes the website address, a fake site cannot reuse it (§6.3).
Authenticator app (M2)	A random secret is shared once with the app. It is offered as a direct link, as copyable text and as speech; a QR code is optional and last. Set-up is complete only when one correct code comes back.	The app computes a 6-digit code from the secret and the current time (TOTP [2]); the server computes the same code and compares. The code changes every 30 seconds.	Only a device holding the secret can produce the code. Each accepted code is recorded so that it cannot be reused (§6.2).

Backup codes support Factor 2. Ten random 20-digit codes are generated at set-up and shown once, in groups of four digits, and only their hashes are stored. If the phone or key is lost, the password plus one backup code is still a full two-factor login, because the codes were given to the user alone and each works once. They are digits only (§7.6).

4.4 Why two factors are used

A password can be guessed, phished or leaked from another site. A device is much harder to obtain remotely, and a passkey's private key is designed never to leave it. Requiring both means that an attacker who only obtains the password still needs access to the second factor to complete a normal login, and an attacker who only obtains the device still does not know the password. A login is two steps and never three, because each extra step is a real cost for a screen-reader user. Studies of blind users report the number of steps, apps and devices as the main difficulty of MFA [12][13].

4.5 Normal login at a glance

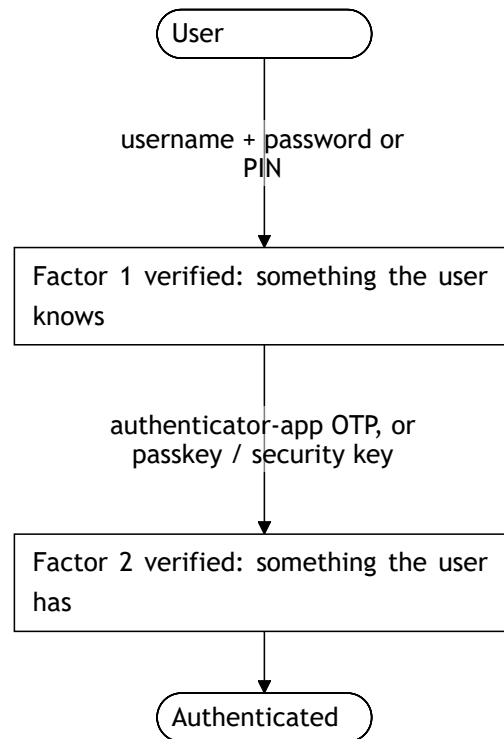


Figure 2. Normal login. After the password or PIN is verified, the user provides the app code or uses the passkey, and is then authenticated. The client announces "Step 1 of 2" and "Step 2 of 2". §5.2 shows the same login in detail.

4.6 Supporting mechanisms

Four supporting mechanisms complete the design. After each authentication step the server creates a fresh random session ID (§4.7). Repeated wrong attempts lock the account for a growing period (§5.3). A forgotten password, a lost device and the loss of everything each have their own recovery route, and the email route is weaker and is kept apart (§5.4). Every security event is written to the audit log (§8.5).

After completing a full MFA login, the user may also choose to remember the browser for 30 days. This is a convenience feature and is not treated as a new authentication factor. It only lets a login that has already proved both factors be reused on the same browser, and it is never enough for a sensitive change.

4.7 Login states and the session

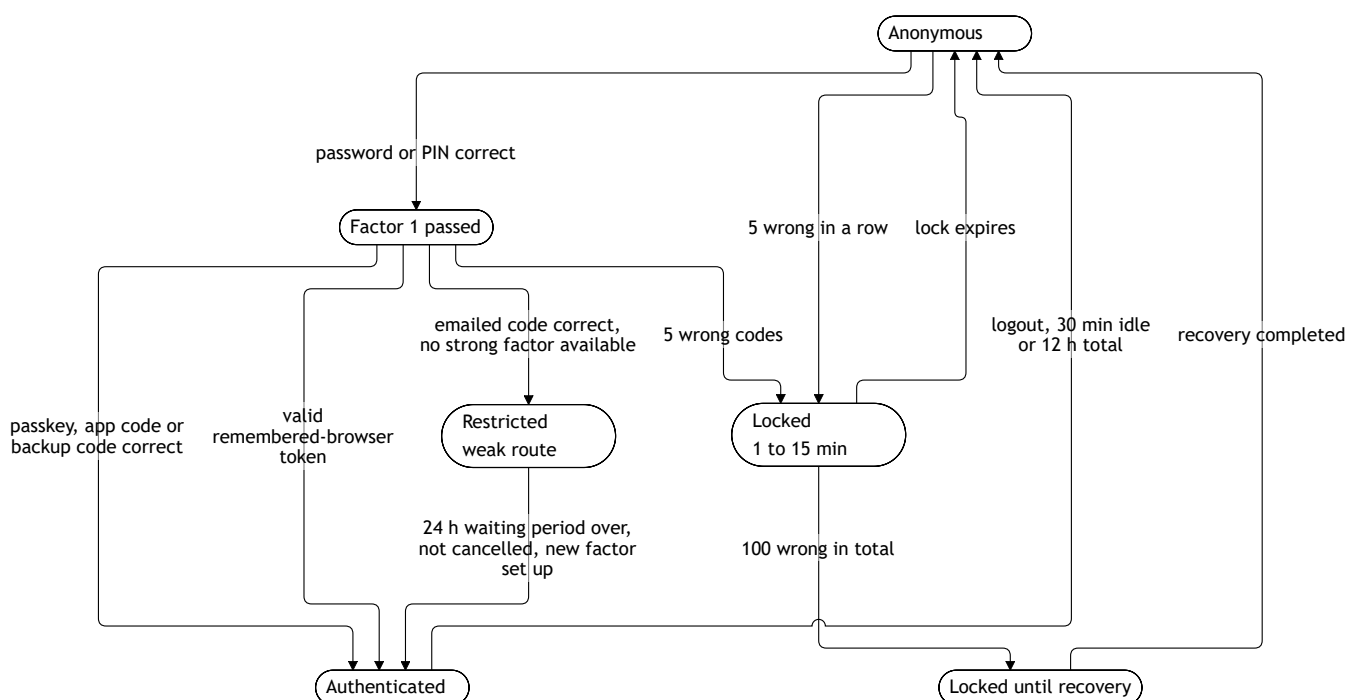


Figure 3. Login states. Every state is held on the server; the client only asks which state it is in. `Authenticated` is reached only after both factors. `Restricted` is a separate state, so it cannot be mistaken for `Authenticated` anywhere in the code. `Locked` can be entered from either step.

After Factor 1 the server creates a session record and gives the browser a random session ID. The session ID is replaced after every step in Figure 3, so an ID captured or planted before a step is useless afterwards. The session expires after 30 minutes without activity or 12 hours in total, and logout deletes the record on the server at once, so a copied ID cannot be used afterwards. Two minutes before the idle limit the client announces a warning and offers a "Stay signed in" button, so no user is timed out without notice.

Sensitive changes need authentication again. Adding or removing a factor, changing the password or email address, or remembering a browser requires the second factor to be proven again if it was last proven more than five minutes ago (SR-9). A remembered browser never satisfies this check.

5. Authentication Flow

Each flow below is designed to be completed by screen reader and keyboard. The reasons for the accessibility choices are given in §7.

5.1 Registration

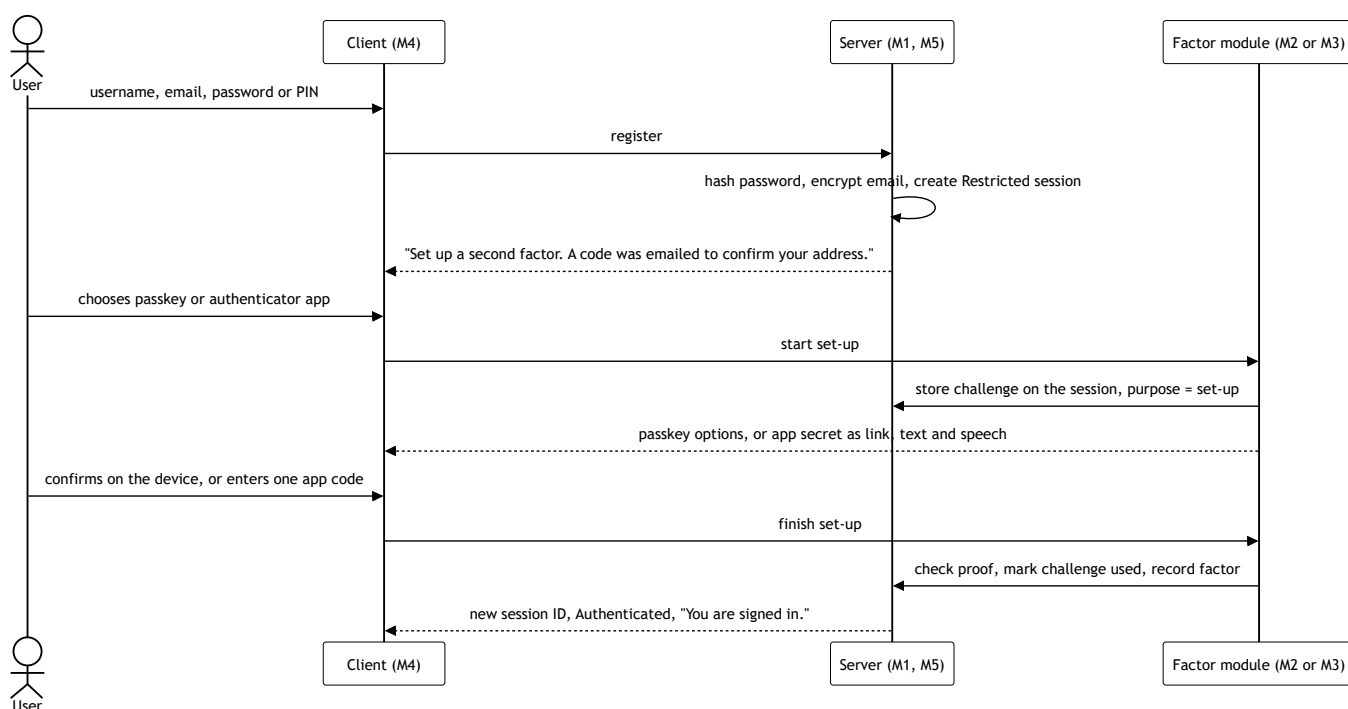


Figure 4. Registration. A new account starts in the `Restricted` state and can do nothing until one strong second factor exists. The email address is confirmed with its own emailed code; until then it is not used for recovery, so nobody can register with another person's address and later "recover" through it. The set-up challenge is stored on the session with its purpose and marked used when answered.

5.2 Normal login

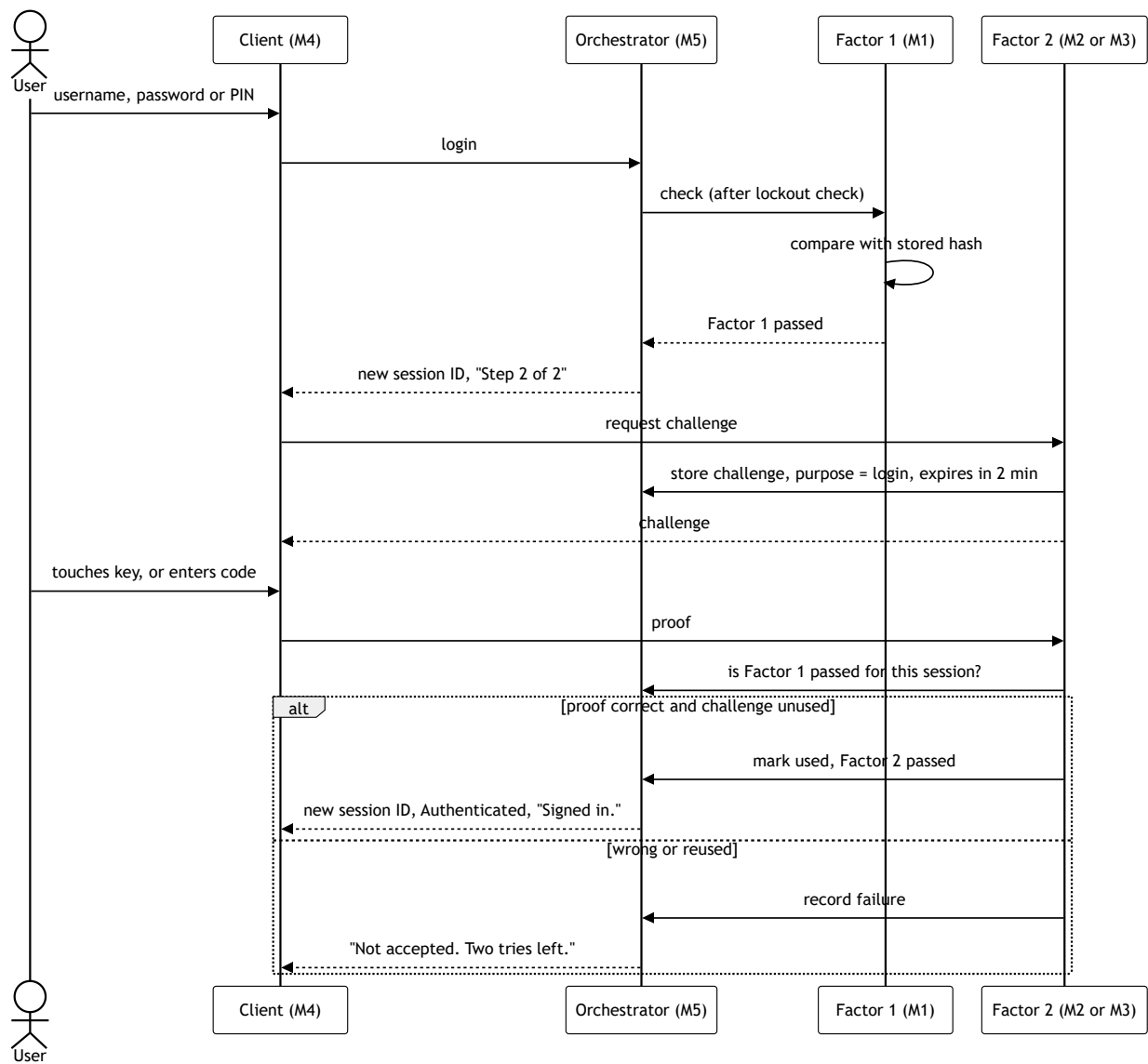


Figure 5. Normal login in detail. The orchestrator checks the lockout state before Factor 1 and checks that Factor 1 has passed before any second-factor proof is examined, so a factor cannot be skipped. The challenge is stored on the session marked "for login" and is marked used before the session is upgraded. The session ID changes after each step.

5.3 Failed login and lockout

Situation	Response
Wrong password, PIN or code	The same message for every case: "Not accepted." with the number of tries left. The message never says which part was wrong or whether the account exists.
5 wrong attempts in a row on one account	Locked for 1 minute; each further 5 doubles the lock, up to 15 minutes. The user is told the exact wait and the other options.
100 wrong attempts in total without a success	Locked until recovery. This cap is what bounds an attacker: with three valid app codes at any moment, a 15-minute lock alone would still allow about 480 guesses a day.
20 wrong attempts from one network address in 15 minutes, on any accounts	That address is slowed down, so one attacker cannot lock out many users.
Many failures from another address	The victim is not locked out, because the lock is counted per account and per address.

5.4 Recovery

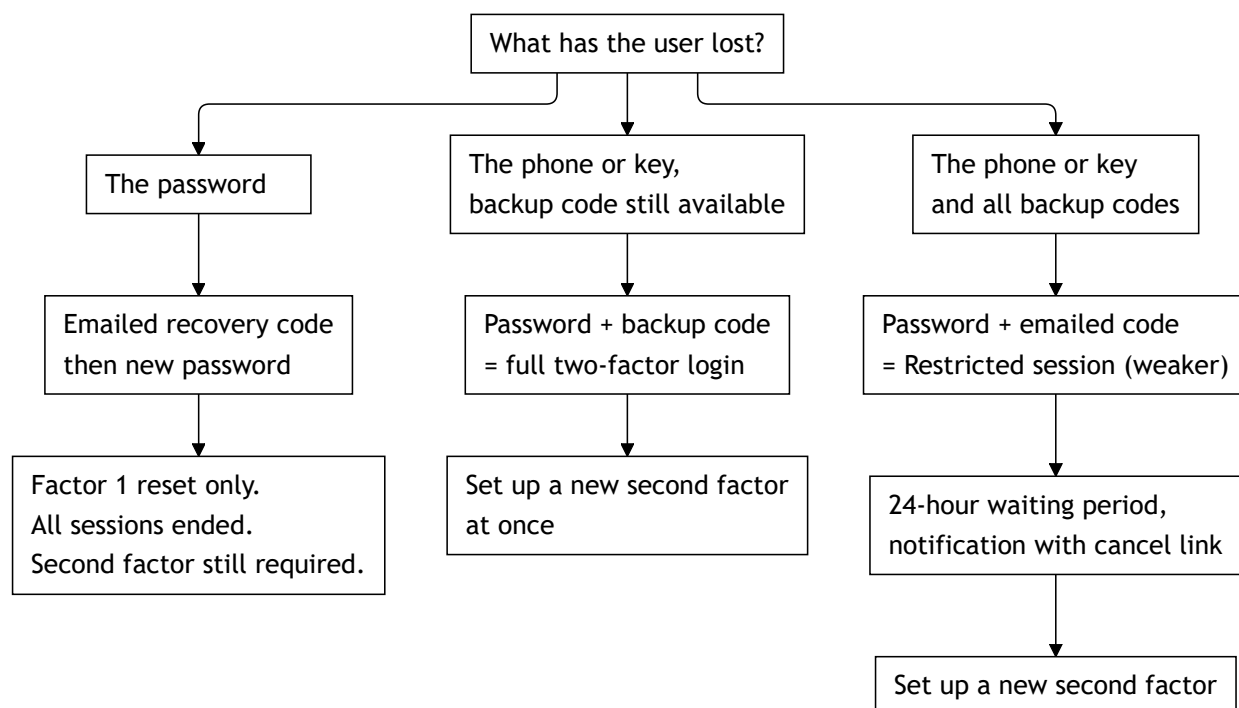


Figure 6. The three recovery cases.

Case	Route	Result
Normal login	Password/PIN + strong second factor	Full authentication
Lost second factor, backup code available	Password + backup code	Full authentication (both factors still proven)
No second factor and no backup code	Password + emailed code opens a restricted session. After a 24-hour waiting period, during which a notification with a cancel link is sent, a new second factor can be set up	Weaker route, not treated as equal to a strong second factor
Forgotten password	Emailed code, then a new password; every session ended	Factor 1 reset only; the second factor is still required at the next login

The emailed-code route is weaker than the normal MFA route. An email can be read by anyone who gains access to the mailbox, and it passes through several servers on the way. It is kept only so that a user who has lost every device is not locked out for ever, and it is limited in three ways. The restricted session can do nothing except ask to set up a new factor. The request waits 24 hours. A notification with a cancel link goes to the confirmed address, so the real user can stop an attacker who holds the password and the mailbox. A user who has lost the device, the backup codes *and* the mailbox cannot recover without help (§11).

A user can also report a factor lost. Marking an app or a passkey as lost stops it working at once; it is deleted 24 hours later unless the user cancels from the notification. Because a lost report also removes the user's own protection, it needs more than the password: a full login, or the password plus a backup code or an emailed code.

6. Security Analysis

This section is the core of the report. An authentication protocol has three objectives. The user must know it is talking to the correct server, the server must know it is talking to the correct user, and a secure session must be established. The protocol must also resist man-in-the-middle, cut-and-paste and replay attacks [9]. The same questions are asked of this design. For each attack, this section states what the attack is, how the attacker performs it, what can happen, and how the design prevents or reduces it.

6.1 Password guessing

Attack. The attacker tries many passwords, either online against the login form or offline against a stolen copy of the database.

How the attacker performs it. Online: a script submits common passwords or passwords leaked from other websites. Offline: the attacker obtains the database and computes the hash of each candidate password until one matches.

What can happen. Factor 1 is defeated. With MFA the attacker still needs the second factor, but a guessed password is the first step of every account takeover.

How the design prevents it. Passwords are checked against a list of common leaked passwords at registration, so the easiest guesses are refused. Online guessing is limited by the lockout rules of §5.3. Offline guessing is made expensive by storing each password as a salted hash and comparing `Hash(password | salt)` with the stored value. The hash function is Argon2id, which is deliberately slow and memory-hungry, so each guess costs the attacker real time [7]. PINs are treated exactly like passwords. Error messages and response times are the same for a wrong password and an unknown username, so the attacker cannot discover which accounts exist.

6.2 Replay attack

Attack. The attacker records a valid authentication message and sends it again later, hoping the server accepts it a second time [9].

How the attacker performs it. The message may be an app code, a backup code, an emailed code, the signed answer to a passkey challenge, or a session ID. It is captured from the network, from a compromised browser, by overhearing a spoken code, or from a stolen device.

What can happen. The attacker passes the second factor without holding the user's device, or reuses an old session.

How the design prevents it. The standard defence against replay is a fresh nonce or a one-time value in every run, so that an old message can be recognised as old [5]. This design applies that rule to every value that could be captured:

Value	Why replay fails
App code (TOTP)	The code depends on the current 30-second time step. The server records the time step of the last accepted code and refuses any code from that step or an earlier one [2].
Emailed code and backup code	Each is marked "used" the moment it is accepted, in the same database operation that checks it was still unused, so two copies of the same request cannot both succeed.
Passkey challenge	The server issues a fresh random challenge for every login, stores it on the session with a 2-minute expiry, and marks it used before it checks the signature. A second copy of the signed answer is refused.
Session ID	Replaced after each step, expires, and is deleted at logout; an old ID is not in the server's records.
Remembered-browser token	Valid on one browser for a limited period only and replaced each time it is used, so a copied token is of little use and can never authorise a sensitive change.

Only the challenge is marked used, never the whole session, so a user who mistypes a code stays at the same step and is not sent back to the start.

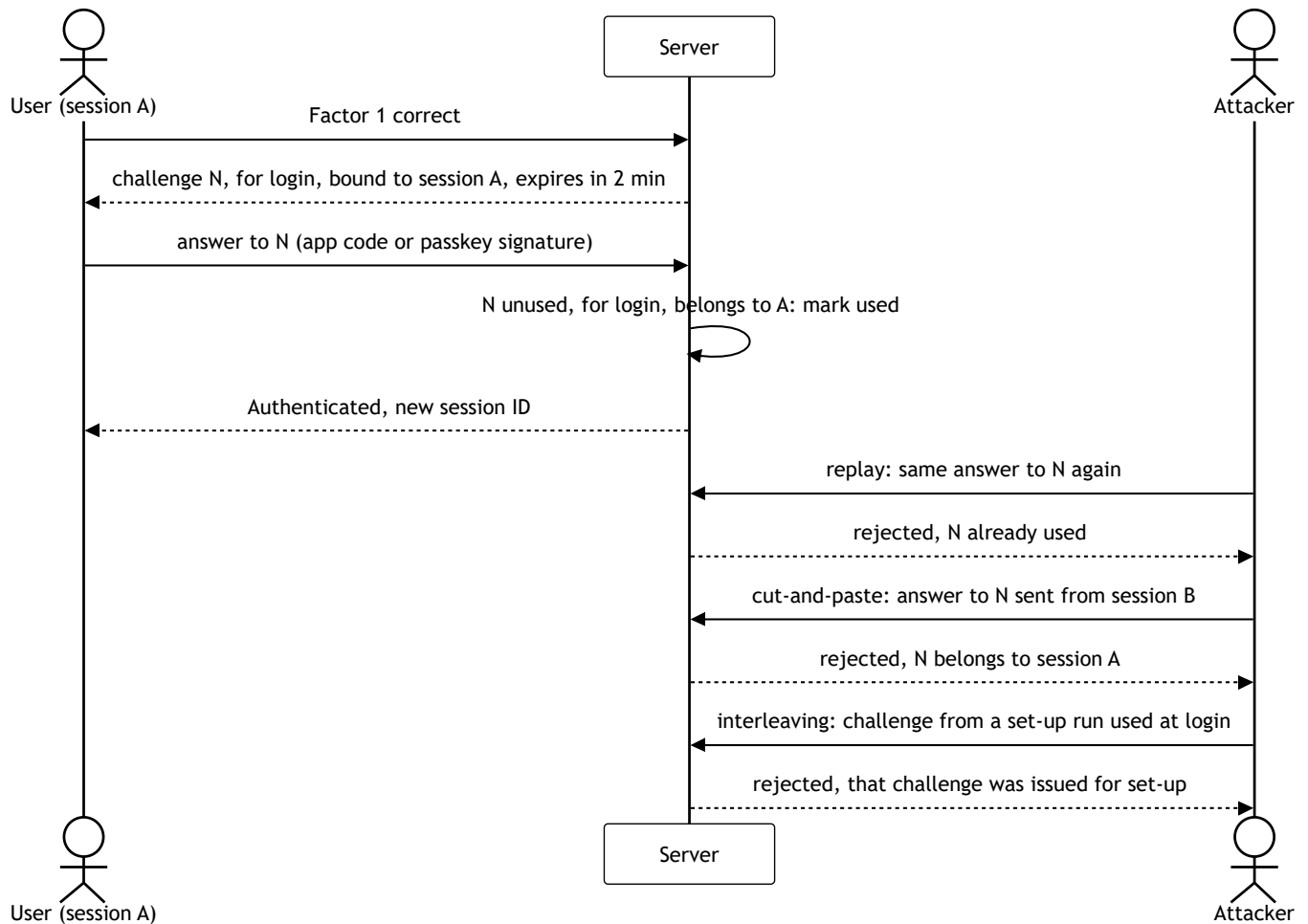


Figure 7. Replay, cut-and-paste and interleaving attempts against the second factor. The same stored challenge record defeats all three: it is single-use (replay), tied to one session (cut-and-paste) and tied to one purpose (interleaving).

6.3 Man-in-the-middle attack

Attack. The attacker positions themselves between the browser and the server and reads, alters or inserts messages, so that there are two separate channels, one between the user and the attacker and one between the attacker and the server [9]. A fake website that forwards the user's input to the real site in real time is the same attack at the application level (phishing relay).

How the attacker performs it. A rogue Wi-Fi access point, a poisoned DNS or ARP entry, a malicious proxy, or a link to a look-alike domain.

What can happen. The attacker learns the password and the current code, takes over the session, or changes what the user sends. This is a total break, because both the short-term session secret and the long-term password are lost.

How the design prevents it. A man-in-the-middle succeeds whenever the parties cannot verify whose public key they are using. The defence is an authenticated public key, embedded in a digital certificate that the other side can check. HTTPS applies exactly that defence [8], and the design meets each condition needed for it:

#	Condition	How the design meets it
C1	The user's browser must be able to verify that it is talking to the real server.	The server's public key is in a certificate signed by a trusted certificate authority (A3); the browser verifies the signature. An attacker should not be able to obtain a valid certificate for the site's name.
C2	Messages must be confidential and their integrity protected in transit.	The HTTPS connection (TLS) encrypts every message and adds an integrity check, so an altered message is detected [8].
C3	The attacker must not be able to force the browser onto plain HTTP.	The server refuses plain HTTP and instructs the browser never to use it for this site [8].
C4	A proof captured at a fake site must be useless at the real one.	The passkey signature covers the website address as well as the challenge. A signature obtained by a look-alike site is for the wrong address, and the real server rejects it. This is why the passkey is the preferred second factor.
C5	Anything captured must be useless after use.	Every code and challenge is single-use and short-lived (§6.2); the session ID changes at every step.

One risk remains. A live fake site can relay an app code or an emailed code to the real site within the code's lifetime, and encryption cannot stop this because the user is typing the code into the attacker's page. The design reduces the damage (single use, 30-second life, login notification, audit log) and removes it entirely for users who log in with a passkey. This is stated as a limitation in §11.

6.4 Cut-and-paste attack

Attack. The attacker takes a valid part of one protocol run and inserts it into another run, so that the server accepts a message that was never created for that run [9].

How the attacker performs it. A challenge issued to one session is answered from another session; a challenge issued for set-up is used for login; a code emailed for password recovery is typed into the login screen; an encrypted secret is copied from one user's database record into another's; a signed passkey response for one operation is submitted for a different one. A classic example is a credential created for one server but delivered to another.

What can happen. The attacker satisfies a check with material that was never issued to them or for that purpose. For example, the attacker could set up a new second factor on the victim's account using a challenge from the attacker's own session.

How the design prevents it. The rule is to bind every message to the run it belongs to, meaning the parties, the session and the purpose. The design binds every value it issues or stores:

#	Condition	How the design meets it
S1	A challenge must fail outside the session and purpose it was issued for.	The challenge is stored on the session record together with its purpose (login, set-up or recovery) and its expiry, and is checked there. Another session cannot present it, and a set-up challenge cannot satisfy login (Figure 7).
S2	A code sent for one purpose must fail for another.	Every emailed code carries a purpose; a recovery code is refused at the login step.
S3	A signed passkey response must not be re-combinable.	The signature covers the challenge, the operation type and the website address together, so changing any one of them breaks the signature [3].
S4	A stored secret must fail if moved to another user's record.	The app secret and the email address are encrypted together with a label naming the user they belong to. A value pasted into another user's record fails the check and is never decrypted.
S5	The server must never answer its own challenge.	The server only issues challenges; it never signs or echoes one, so there is nothing an attacker can reflect back.

6.5 Interleaving attack

Attack. The attacker runs two protocol sessions at the same time and uses a message from one to answer the other [9]. Such attacks succeed when the same secret is reused across servers and a value that should be fresh is not random.

How the attacker performs it. While the real user is setting up a factor or logging in, the attacker opens a second session and tries to feed the server's challenge from one session into the other, or reuses a value the server issued for a different purpose.

What can happen. The attacker completes a step of their own session with a proof that the real user produced, and reaches a state they should not reach.

How the design prevents it. Every challenge is a fresh random value that is never reused and never derived from a predictable quantity. It is bound to one session and one purpose (S1) and expires after two minutes, and the server never issues a value that could serve as an answer in another run (S5). Two simultaneous submissions for one session are handled one at a time, so exactly one can consume the challenge. The bottom of Figure 7 shows an interleaving attempt being rejected.

6.6 Session misuse

Attack. The attacker uses the user's session instead of authenticating: by stealing the session ID (hijacking), by planting a known session ID before the user logs in (fixation), or by making the user's browser send a request without the user's intent (cross-site request forgery).

How the attacker performs it. Script injected into the page reads the session ID; a link sets the ID before login; a malicious page submits a form to the login server.

What can happen. The attacker acts as the signed-in user without either factor.

How the design prevents it. The session ID is a long random value, and the server stores only its hash, so a stolen database does not allow one to be forged. It cannot be read by page scripts and is sent only over HTTPS. The server never accepts a session ID it did not issue, and it issues a new ID after Factor 1 and again after Factor 2, so a planted ID is

discarded. Requests that change anything must carry a token that only the real page has. Sessions expire after 30 minutes idle or 12 hours in total, and logout deletes the server record immediately, so a copied ID has no value afterwards.

6.7 Bypassing the second factor

Attack. The attacker tries to reach the signed-in state after only the password: by calling the "signed in" page directly, by choosing the weak email route, by using a remembered browser for a sensitive change, or by reporting the real user's factors lost.

How the attacker performs it. Requests are sent directly to the server, out of order or with a chosen route, rather than through the intended screens.

What can happen. MFA is reduced to a single factor.

How the design prevents it. The state of every session lives on the server (Figure 3), and every request is checked against it before anything else happens. A second-factor request from a session that has not passed Factor 1 is refused, and a request for a signed-in page from a `Factor 1 passed` session is refused. The client never decides on its own that a user is signed in. The email route leads only to `Restricted`, which cannot read account data or change anything, and can add a factor only after the 24-hour waiting period with a cancel link. Sensitive changes need the second factor again (§4.7), and a remembered browser never counts for them. Reporting a factor lost needs more than the password, so an attacker with only the password cannot strip the user's protection.

6.8 Eavesdropping on spoken secrets

Attack. A secret that the screen reader speaks aloud, such as the app secret at set-up, a backup code or a code being typed, is overheard by someone nearby. Blind users name this as a risk they face every day [11][13].

How the attacker performs it. Simply by being within earshot, in a shared office, a library or a bus.

What can happen. The attacker learns a backup code or the app secret, which is the second factor itself.

How the design reduces it. Secrets are never spoken automatically. Before a secret the client says "The next content is secret. Use headphones if you are not alone" and offers a silent alternative, either copying to the clipboard or opening the app directly by a link, so that the secret never has to be spoken or typed. A "private place" setting turns off all optional speech and hides codes on screen until a button reveals them.

6.9 Summary

Attack	Main protection	Requirement
Password guessing	Common-password list; salted Argon2id hashes; logout	SR-5, SR-6
Replay	Fresh single-use challenge or code; time-step record; session ID replaced	SR-3, SR-7
Man-in-the-middle	Server authenticated by certificate; encrypted, integrity-checked connection; no plain HTTP; passkey signature includes the site address	SR-1
Cut-and-paste	Challenge bound to session and purpose; codes bound to purpose; signature covers all fields; stored secrets bound to their owner	SR-4, SR-5
Interleaving	Fresh random challenge per run; one purpose; one session; never reflected	SR-3, SR-4
Session misuse	Random hashed ID; new ID at each step; HTTPS only; expiry and logout	SR-7
Bypassing the second factor	Server-side state checked on every request; weak route restricted; second factor re-proven for changes	SR-2, SR-9, SR-10
Eavesdropping on audio	Secrets never spoken automatically; silent alternatives	AR-7

What the design does not prevent is listed in §11.

7. Accessibility Design

This section explains, from the user's point of view, how each part of the login can be used without sight, and why each choice was made.

7.1 Screen readers

Every screen has one heading, a "skip to content" link, one main action and one spoken prompt. When a step completes or fails, the result is announced automatically without moving the cursor, so the user does not lose their place. Only lockout and security warnings interrupt the reader; everything else waits for the current sentence to finish. Progress is spoken as "Step 1 of 2" and "Step 2 of 2". This matters because a blind user hears the page one element at a time, and unexpected focus jumps and interruptions are the main way such users lose their place.

7.2 Keyboard-only navigation

Every control can be reached with the Tab key in reading order, the focused control is always visibly outlined, and every action is a labelled button ("Continue", "Copy the key", "Read the key aloud", "Resend", "Stay signed in"). There are no single-letter shortcuts, because they are triggered by accident when a voice-control user dictates text. Screen-reader users do not use a mouse, so a control that only a mouse can reach does not exist for them.

7.3 Braille displays and deaf-blind users

Every message that is spoken is also present on the page as text, and stays there, because a braille display shows the text of the page and cannot show a sound. Codes and keys are shown in groups of four digits so that they can be read from a braille line. Braille users may have speech switched off, and deaf-blind users cannot hear at all, so any audio-only step would exclude them (A9).

7.4 Low vision: large text and high contrast

Text can be enlarged to 200 % and pages reflow at 400 % zoom without sideways scrolling. A high-contrast theme and a dark theme are available and the choice is remembered. Every button is large enough to hit with a finger or an unsteady pointer. Low-vision users often use a mouse, a finger or a stylus as well as the keyboard, and a study participant with low vision asked for exactly this dark mode [13].

7.5 Audio output and text alternatives

Speech output is optional and off by default. When it is on, it reads the server's announcements for users who have no screen reader and can repeat any code slowly or digit by digit. It is never the only channel, because the same text is always on the page. Some users have no screen reader installed on a shared computer, but audio alone would exclude braille users, so text remains the base.

Secrets are never read aloud automatically. Before a secret the client warns "Use headphones if you are not alone" and offers a silent alternative, because a spoken secret can be overheard (§6.8).

7.6 Codes and secrets

The app secret is offered first as a link that opens the authenticator app directly, then as text to copy, then read aloud in NATO-alphabet groups. A QR code is optional and last. QR codes are not used as the only method for setting up an authenticator because scanning a QR code normally requires sight, and typing 32 characters from speech is error-prone.

Code fields accept paste and autofill, allow spaces and hyphens, and are read to the screen reader digit by digit. Blind users specifically asked for this, because screen readers otherwise read "123456" as one large number [11]. Backup codes are digits only, in groups of four, because letters such as B, D, P and T sound alike when spoken.

For passkeys, the page says "Registering a security key for Cryptix" before the browser's own dialog opens, because some screen readers cannot read that dialog [14]. A fingerprint is never required, and a device PIN is enough.

7.7 Error messages and time limits

Every error is a full sentence, both announced and shown next to the field, naming what happened and what to do next: "That code has expired. Choose Resend to get a new one." It is never given by colour or an icon alone. No countdown is displayed and no screen moves on by itself; two minutes before the session ends the client says "Your session will end in two minutes" and offers a "Stay signed in" button. A message that is only red, or a timer, is invisible to a screen-reader user, and a screen-reader user needs more time to find and fill a field.

7.8 What the system never does

The system has no visual CAPTCHA (an audio CAPTCHA is not an acceptable substitute either [11]), no QR-code-only step, no instruction given by colour alone, no drag-and-drop-only interaction and no screen that moves on automatically. Each of these would make a step impossible without sight or a mouse.

The screens are Register, Choose second factor, Set up, Backup codes, Login step 1, Login step 2, Signed in, Manage factors and Recovery. The client always asks the server which state the session is in and never decides on its own.

8. Data and Password Protection

8.1 Hashing

If the server only ever needs to check a value that the user will present again, it stores a hash. Passwords, PINs and backup codes are stored as salted hashes, which means the original values are not stored in the database, and a stolen database

gives the attacker only something to guess against. The salt means that two users with the same password have different hashes. The hash function is Argon2id, which is slow by design so that offline guessing is expensive [7].

8.2 Encryption

Some values must be recovered later: the app secret is needed to compute the expected code, and the email address is needed to send mail. These values are stored encrypted, and the key is kept outside the database in a protected file (A5). A stolen database is then unreadable without the key. The encryption also checks a label naming the user the value belongs to, so a value copied into another user's record fails to decrypt (§6.4, S4).

8.3 Digital signatures for passkeys

The passkey's private key is never stored by the server at all; only the public key is. At login the server verifies the device's signature over the challenge and the website address with that public key. This is public-key authentication: a signature proves that the message came from the holder of the private key and cannot be forged [3].

8.4 Data model

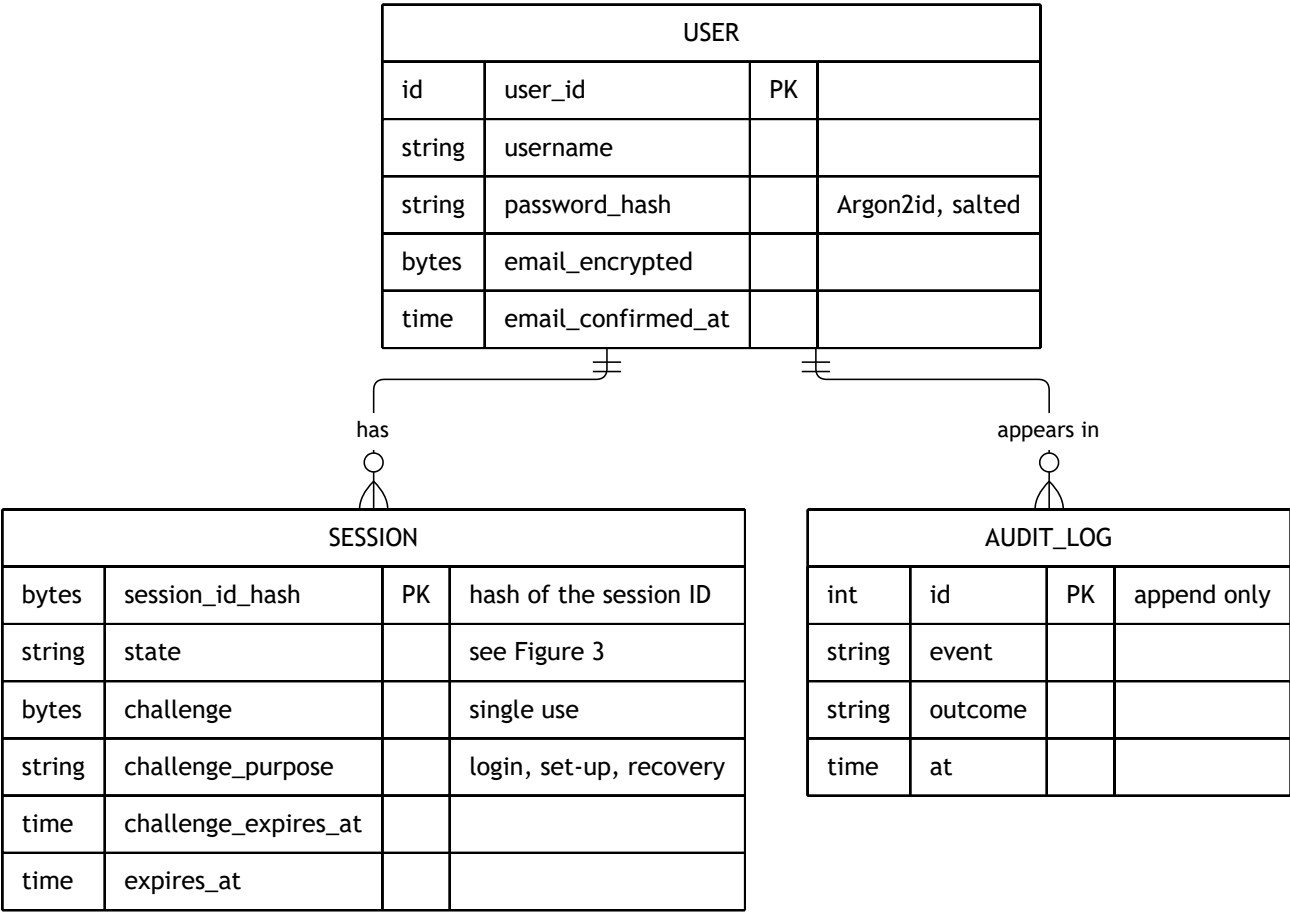


Figure 8. Users, sessions and the audit log.

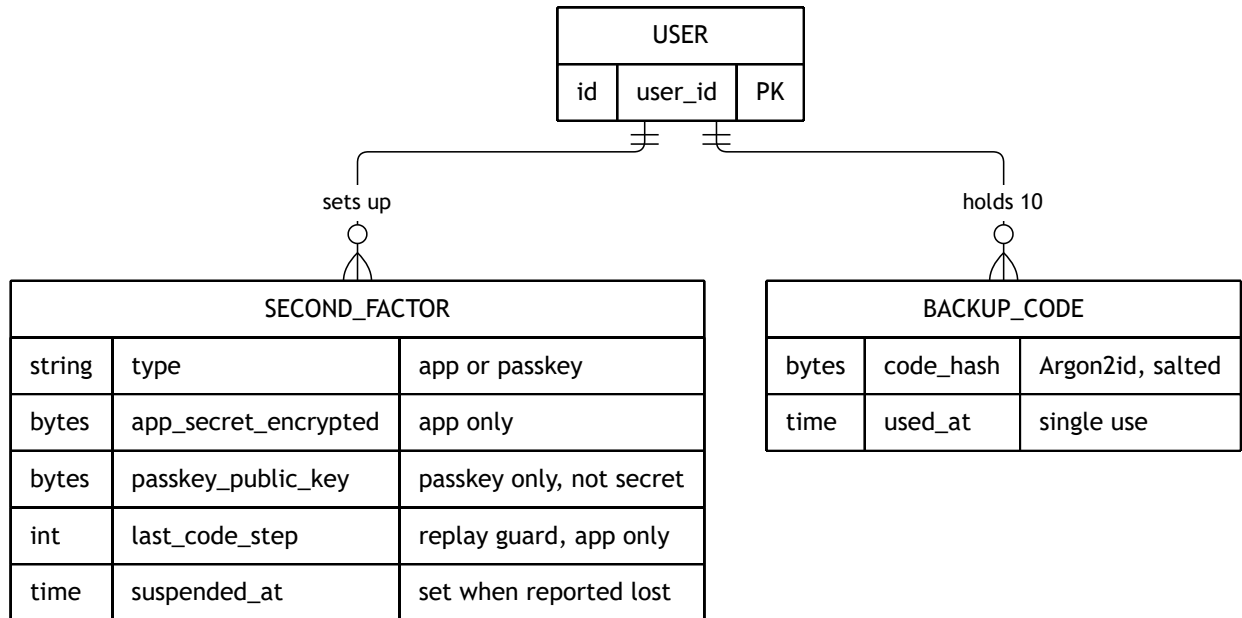


Figure 9. Second factors and backup codes.

Entity	Purpose	What it never contains
User	The account: username, password hash, encrypted email address	The password or the readable email address
Second factor	One record per authenticator app or passkey: the encrypted app secret, or the passkey public key; the last accepted time step; whether it was reported lost	A private key
Session	The login state of one browser session, and the current single-use challenge with its purpose and expiry. This is the record that defeats replay and cut-and-paste (§6.2, §6.4)	The session ID in readable form (only its hash)
Backup code	Ten hashed codes per user, each marked when used	A readable code
Audit log	One record per security event: what happened, the outcome, the time and the address. Records can be added but not changed or deleted	Any secret

8.5 Keys, audit log and retention

The encryption keys and the secret used for keyed hashes are kept in a file that only the server process can read, and they are never written to the database or to backups. The audit log makes every login, set-up, failure, lockout and recovery event traceable afterwards [5]. Data is not kept longer than needed. Sessions are deleted at logout or expiry, emailed codes an hour after they expire, and audit records after 90 days. No voice data, fingerprints or faces are ever collected (A9).

9. Main Design Decisions

ID	Decision	Reason	Source	Ref
D1	A web-based accessible client: plain HTML, no native app and no front-end framework.	Screen readers already work in browsers and read standard HTML controls best; passkeys are built into the browser.	M4	[3][4]
D2	Password or PIN as Factor 1, stored as a salted hash.	Something the user knows; a PIN suits keypad entry by touch; hashing means the password is never stored.	M1	[5][7]
D3	Authenticator-app OTP or passkey / security key as Factor 2, with backup codes; passkey preferred.	Something the user has. The passkey signature includes the website address, so it resists fake sites; the app works on any phone; backup codes cover loss.	M2, M3	[2][3]
D4	Two different factors, exactly one login of two steps.	Knowing the password alone is not enough; a third step adds cost for screen-reader users without adding a new kind of proof.	Group	[1][12][13]
D5	A fresh challenge or one-time value for every second-factor check, bound to the session and purpose, accepted once.	The standard defence against replay, cut-and-paste and interleaving attacks, applied to every value the server issues.	M5	[9]
D6	All authentication state is held on the server; the client only asks what state it is in.	A factor cannot be skipped by calling a page directly, and the client cannot be tricked into believing a user is signed in.	M5	[5]

ID	Decision	Reason	Source	Ref
D7	Secure session management: random session ID, replaced after each step, expiry, deleted at logout.	Defeats session fixation and hijacking, and makes anything captured before a step useless afterwards.	M1	[6]
D8	Restricted recovery: the emailed code opens only a restricted session, and a new factor can be added only after a 24-hour waiting period with a cancel link.	Email is weaker than a device; the waiting period gives the real user time to cancel if an attacker holds the password and the mailbox.	M2, M5	
D9	Accessibility-first interface: secret as link, text and speech; codes read digit by digit; no timers; no single-letter shortcuts; large text and high contrast.	Nothing may depend on sight, memory or speed; these are the specific difficulties blind and low-vision users report.	M2, M4	[4] [11]
D10	Hashing for values that are only checked; encryption with keys outside the database for values that must be recovered; an append-only audit log.	A stolen database alone is then not enough, and every security event can be traced.	M1, M2, M5	[5][7]

10. Testing

No test has been run yet, because the design has not been implemented. Every row below is marked Planned, and a row will be marked Verified only when it has passed on the implementation.

10.1 Security tests

ID	Test	Expected result	Checks	Status
ST-1	Correct password and correct app code; correct password and passkey	Signed in; session ID changed twice; audit record written	FR-2, SR-7	Planned
ST-2	Wrong password five times, then a sixth	Same message each time with tries left; sixth attempt locked for 1 minute	SR-6, SR-8	Planned
ST-3	Wrong app code, then the correct one	Error, then signed in without returning to step 1	SR-3	Planned
ST-4	The same app code sent twice; the same backup code sent twice	Second attempt refused	SR-3	Planned
ST-5	The same signed passkey answer sent twice; two copies sent at the same instant	Second refused; of two simultaneous copies exactly one succeeds	SR-3	Planned
ST-6	Cut-and-paste: a challenge issued to session A answered from session B; a recovery code typed at the login step	Both refused	SR-4	Planned
ST-7	Interleaving: a set-up challenge used at login while both sessions are open	Refused	SR-4	Planned
ST-8	Attempt to skip Factor 2: a signed-in page requested after Factor 1 only; a second-factor request from a session that has not passed Factor 1	Both refused	SR-2	Planned
ST-9	Session expiry, logout, and a session ID planted before login	Expired and logged-out sessions refused; the planted ID is not the ID used after login	SR-7	Planned
ST-10	Man-in-the-middle: a proxy with an untrusted certificate; an <code>http://</code> address; a passkey login from a look-alike site address	Browser refuses the connection; plain HTTP never reaches the login page; the signature is rejected	SR-1	Planned
ST-11	Recovery: forgotten password; lost device with a backup code; emailed-code route	Password reset ends all sessions and still needs Factor 2; backup-code login is a full login; the email route reaches only the restricted state, cannot add a factor before the waiting period, and the cancel link stops it	FR-5, FR-6, SR-10	Planned

10.2 Accessibility tests

ID	Test	Expected result	Checks	Status
AT-1	Register, set up, log in and recover with NVDA and the monitor switched off	Every flow completed; every result announced; no secret spoken without a keystroke	AR-1, AR-4, AR-7	Planned
AT-2	The same flows with the keyboard only, no mouse	Every control reachable in order; focus always visible	AR-3, AR-9	Planned
AT-3	The same flows with VoiceOver (macOS, iOS) and TalkBack (Android); with a braille display and speech muted; by voice control alone	Every flow completed; every message readable as text	AR-3, AR-4	Planned

ID	Test	Expected result	Checks	Status
AT-4	Every error message: wrong code, expired code, lockout, cancelled passkey prompt	A full sentence announced and shown, naming the fix; nothing conveyed by colour alone	AR-5	Planned
AT-5	Passkey and OTP set-up and login: secret obtained as link, text and speech; codes pasted and autofilled; fingerprint fails then device PIN	Every route works without a QR code; codes read digit by digit	AR-2	Planned
AT-6	Automated accessibility scan; zoom at 200 % and 400 %; high-contrast theme	No automatically detectable WCAG 2.2 AA failures; no sideways scrolling; buttons usable	AR-8	Planned

Testing with blind and low-vision participants is planned but has not been performed (§11).

11. Limitations and Future Work

The system is not claimed to be completely secure. The following limitations follow from the design itself:

- A live phishing website can relay an app code or an emailed code to the real site before it expires (§6.3). The passkey option is not affected, because its signature is tied to the website address, so users are encouraged to set up a passkey.
- A malicious browser extension or screen-reader add-on on the user's own device cannot be stopped by the server (A7). The passkey's private key is the one secret that stays on the device.
- Email recovery is weaker than normal MFA. It is restricted, delayed by 24 hours and cancellable, but it is still weaker than a device-based second factor. An attacker who holds both the password and the mailbox can also start the waiting period or suspend the real user's factors for up to 24 hours. The cancel link and the backup codes limit this.
- The project does not perform real-world identity verification. A user who loses the device, the backup codes and the mailbox cannot recover alone, and a manual identity check is out of scope (A8).
- Encryption keys are in a file on the server, not in dedicated hardware (A5). A full compromise of the server exposes them.
- The system has not yet been tested with visually impaired users. The accessibility design is based on the WCAG guidelines and on published studies of blind and low-vision users [11][12][13], but no test in §10 has been run yet.

Three items of future work follow from these limitations: testing with blind and low-vision participants, a second independent recovery channel to strengthen the email route, and the changes needed to run the service on more than one server.

12. Conclusion

This report has designed a multi-factor authentication system for visually impaired users. A normal login needs two different kinds of proof: a password or PIN that the user knows, and a device that the user has, holding an authenticator app or a passkey. The whole process, from registration to recovery, can be completed with a screen reader and a keyboard, and no step depends on sight.

The security analysis considered password guessing, replay, man-in-the-middle, cut-and-paste and interleaving attacks, session misuse and bypassing of the second factor. The main protections are HTTPS with server certificates, fresh single-use challenges that are bound to one session and one purpose, salted password hashing, server-side session state with a session ID that changes after each step, and a restricted recovery route that is clearly weaker than the normal second factor.

The design has limitations. A live phishing site can still relay a code before it expires, a compromised user device cannot be protected by the server, email recovery is weaker than normal MFA, and the system has not yet been implemented or tested with visually impaired users. These are stated openly, and the test plan separates what has been designed from what has been verified.

References

1. NIST. *Digital Identity Guidelines: Authentication and Authenticator Management*. SP 800-63B, Revision 4, 2025. <https://pages.nist.gov/800-63-4/sp800-63b.html>
2. M'Raihi, D. et al. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238, IETF, 2011. <https://www.rfc-editor.org/rfc/rfc6238>
3. W3C. *Web Authentication: An API for Accessing Public Key Credentials, Level 3*. <https://www.w3.org/TR/webauthn-3/>
4. W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, 2023. <https://www.w3.org/TR/WCAG22/>
5. OWASP. *Authentication Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

6. OWASP. *Session Management Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

7. Biryukov, A., Dinu, D., Khovratovich, D., Josefsson, S. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106, IETF, 2021. <https://www.rfc-editor.org/rfc/rfc9106>

8. Rescorla, E. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, IETF, 2018; Hodges, J., Jackson, C., Barth, A. *HTTP Strict Transport Security (HSTS)*. RFC 6797, IETF, 2012.

9. Shirey, R. *Internet Security Glossary, Version 2*. RFC 4949, IETF, 2007. <https://www.rfc-editor.org/rfc/rfc4949>

10. WebAIM. *Screen Reader User Survey #10*. 2024. <https://webaim.org/projects/screenreadersurvey10/>

11. Usmani, M.F., Buhasio, R.K., Manduri, S.M., Vishwakarma, S.S. *Accessible Two Factor Authentication for Users with Visual Impairments*. Human-Machine Interaction course report, Frankfurt University of Applied Sciences, 2021.

12. Toussaint, C., Chateau, B., Gourio-Jewell, P.-G., Bonnefoy, É., Louveton, N. *Inclusive by design: Developing Barrier-Free Authentication for Blind and Low Vision Users through the ALIAS Project*. Proc. ECCE 2025. <https://doi.org/10.1145/3746175.3746210>

13. Bhole, P.V., Li, Z., Bokolia, S., Oh, T., Tigwell, G.W., Peiris, R.L. *Haptic2FA: Haptics-Based Accessible Two-Factor Authentication for Blind and Low Vision People*. Proc. ACM Hum.-Comput. Interact. 8 (MHCI), 2024. <https://doi.org/10.1145/3676509>

14. Akanda, M.M.R.R., Mahdad, A.T., Saxena, N. *Broken Access: On the Challenges of Screen Reader Assisted Two-Factor and Passwordless Authentication*. Proc. ACM Web Conference 2025. <https://doi.org/10.1145/3696410.3714579>

Appendix A. Combining the five individual designs

Point of difference	What the individual designs said	Decision
Where sessions live	M1: a table in the database. M5: a separate in-memory store.	Database table (one data store).
Email address storage	M1: hashed. M2, M5: must be recoverable to send mail.	Encrypted, with a keyed hash for lookup (§8.2).
Lockout numbers	M1: unspecified. M2: 5 attempts then a 15-minute lock, 100 in total. M5: 1-minute lock, then 14 minutes.	\$5.3.
Timers on each step	M4, M5: a timer per step. M1, M2: none.	None per step; only the session warning (AR-6).
Which factor combinations are allowed	M3: Factor 1 plus one of the two. M1: "either or both".	Factor 1 plus exactly one strong factor (D4).
Number of login steps	M4: three steps every login.	Two steps (D4).
"Trusted device"	M3: a factor.	A convenience, not a factor (§4.6).
Braille and deaf-blind users	M4: out of scope. M2, M3: served from the page text.	Served from the page text (A9).
Speech output	M4: opt-in, and switched off automatically if a screen reader is detected.	Opt-in only; a web page cannot detect a screen reader.

Appendix B. API summary

All calls use HTTPS. Calls that change anything require the session cookie and a request token. Every response tells the client the current state, the next step, and a sentence to be read aloud.

Purpose	Endpoints	Owner
Register; confirm email; resend confirmation	POST /register, POST /email/verify, POST /email/resend	M1
Log in (Factor 1); log out; current state	POST /login, POST /logout, GET /auth/status	M1, M5
Authenticator app: set up, confirm, verify, report lost, remove	POST /mfa/totp/enroll, /confirm, /verify, /lost; DELETE /mfa/totp	M2
Emailed code: send, verify; restricted waiting period	POST /mfa/email/send, POST /mfa/email/verify, POST /mfa/restricted/hold	M2, M5
Backup codes: generate, verify	POST /mfa/backup/generate, POST /mfa/backup/verify	M2
Passkey: register, log in, list, report lost, remove	POST /webauthn/register/options and /verify, POST /webauthn/login/options and /verify, GET /webauthn/credentials, POST .../lost, DELETE .../{id}	M3
Remember / forget this browser	POST /device/remember, POST /device/verify, DELETE /device	M3
Password recovery; cancel a waiting period or deletion	POST /recovery/start, POST /recovery/verify, GET /mfa/cancel/{token}	M5